Operating Systems - Examples & Problem Set

This document contains all the extracted examples, commands, and scheduling problems organized day-wise. Unsolved examples from the original notes have been completely solved and explained.

Day 1: Basic Linux File System Commands

Ex 1: Directory Navigation Basics

- Check the present working directory: pwd
- Move to home directory: cd /home
- Move to root directory: cd /
- Every time, check location using pwd.

Ex 2: Navigating to Specific Folders

- Move to the faculty folder: cd /home/faculty
- Check path: pwd
- Clear the screen: clear

Ex 3: Listing Contents

- Ensure your working directory is the faculty folder.
- List contents: ls

Ex 4: Creating Folders

- Create a folder named OS in your login directory: mkdir /home/yourlogin/OS
- Verify creation: ls

Ex 5: Observing Home Directories

- Navigate to home: cd /home
- List contents: ls
- *Observation: Every login gets a separate folder in the home directory.*

Ex 6: Folder Permissions

- Change directory to another group folder.
- *Observation: You do not have access permission for other users' folders, only your own.*

Ex 7: Root Directory Contents

- Move to root: cd /
- Check content of root using: ls

Ex 8: Nested Directories

- Move to the OS folder created earlier.
- Create two more folders: mkdir day1 day2
- Check using: ls

Day 2: Environment Variables & Paths

Ex 9: Prompt String (PS1)

- Observe the current value of the PS1 variable: echo "$PS1"
- Change the value of PS1 variable: export PS1="\W >>" (or any value of your choice).
- Observe the changed PROMPT by navigating: cd /, cd /home, cd /home/faculty/OS.

Ex 10: Environment Variables

- Run the env command to see different environment variables.
- Print the values of specific variables: echo $PATH, echo $HOME, echo $USER

Ex 11: Relative Paths

- Ensure pwd is ~ (home).
- Create a folder using relative path: mkdir ./planets and check with ls.
- Create folders inside planets without changing pwd: mkdir ./planets/earth ./planets/jupiter
- Check creation: ls ./planets
- Ensure pwd is ~. Create a folder Moon inside earth: mkdir ./planets/earth/Moon

Ex 12: Absolute Paths

- Create a folder day2 in OS using absolute path: mkdir /home/faculty/OS/day2
- Check if day2 is created: ls /home/faculty/OS

Ex 13: Parent Directory Navigation

- Move to the previous folder using cd .. until you reach /.
- Go to the Moon folder using a single cd command with absolute or relative path.
- Go to /home folder using a single cd with a relative path (e.g., cd ../../..).

Day 3: Process Scheduling Problems

1. First Come First Serve (FCFS)

Problem: Calculate Average Wait time (W_t) and Turnaround time (T_{at}) using FCFS.

|

| **Process** | **Arrival Time** | **CPU-burst-time** |

| P1 | 0 | 5ms |

| P2 | 1 | 3ms |

| P3 | 4 | 4ms |

| P4 | 3 | 1ms |

Solution:

- W_t = Start time - Arrival time
  - $W_{t1} = 0 - 0 = 0$
  - $W_{t2} = 5 - 1 = 4$
  - $W_{t3} = 9 - 4 = 5$
  - $W_{t4} = 8 - 3 = 5$
  - **Average** W_t = $\frac{0+4+5+5}{4} = 3.5 \text{ ms}$
- T_{at} = Wait time + CPU-burst-time
  - $T_{at1} = 0 + 5 = 5$
  - $T_{at2} = 4 + 3 = 7$
  - $T_{at3} = 5 + 4 = 9$
  - $T_{at4} = 5 + 1 = 6$
  - **Average** T_{at} = $\frac{5+7+9+6}{4} = 6.75 \text{ ms}$

2. Shortest Job First (SJF) - Non-Preemptive

Problem: Calculate Average W_t and T_{at} using Non-Preemptive SJF.

| **Process** | **Arrival Time** | **CPU-burst-time** |

| P1 | 0 | 7ms |

| P2 | 1 | 4ms |

| P3 | 2 | 2ms |

Solution:

- $W_{t1} = 0 - 0 = 0$
- $W_{t2} = 9 - 1 = 8$
- $W_{t3} = 7 - 2 = 5$

- **Average** W_t = $\frac{0+8+5}{3} = 4.33 \text{ ms}$
- $T_{at1} = 0 + 7 = 7$
- $T_{at2} = 8 + 4 = 12$
- $T_{at3} = 5 + 2 = 7$
- **Average** T_{at} = $\frac{7+12+7}{3} = 8.66 \text{ ms}$

3. Preemptive SJF (Shortest Remaining Time First)

Problem: Calculate Average W_t and T_{at} .

| **Process** | **Arrival Time** | **CPU-burst-time** |

| P1 | 0 | 7ms |

| P2 | 1 | 4ms |

| P3 | 2 | 2ms |

| P4 | 3 | 5ms |

Solution:

- W_t = (Start - Arrival) + $\sum(\text{Resume} - \text{Preempt})$
  - $W_{t1} = (0 - 0) + (12 - 1) = 11$
  - $W_{t2} = (1 - 1) + (4 - 2) = 2$
  - $W_{t3} = 2 - 2 = 0$
  - $W_{t4} = 7 - 3 = 4$
  - **Average** W_t = $\frac{11+2+0+4}{4} = 4.25 \text{ ms}$
- $T_{at} = W_t + \text{CPU-burst-time}$
  - $T_{at1} = 11 + 7 = 18$
  - $T_{at2} = 2 + 4 = 6$
  - $T_{at3} = 0 + 2 = 2$
  - $T_{at4} = 4 + 5 = 9$

- **Average** T_{at} = $\frac{18+6+2+9}{4} = 8.75 \text{ ms}$

4. Non-Preemptive Priority

Problem: Calculate Average W_t and T_{at} (Priority: 10 is highest, 1 is lowest).

| **Process** | **Arrival Time** | **CPU-burst-time** | **Priority** |

| P1 | 0 | 7 | 4 |

| P2 | 0 | 4 | 7 |

| P3 | 3 | 5 | 9 |

| P4 | 1 | 2 | 1 |

Solution:

- $W_{t1} = 9 - 0 = 9$
- $W_{t2} = 0 - 0 = 0$
- $W_{t3} = 4 - 3 = 1$
- $W_{t4} = 16 - 1 = 15$
- **Average** W_t = $\frac{9+0+1+15}{4} = 6.25 \text{ ms}$
- $T_{at1} = 9 + 7 = 16$
- $T_{at2} = 0 + 4 = 4$
- $T_{at3} = 1 + 5 = 6$
- $T_{at4} = 15 + 2 = 17$
- **Average** T_{at} = $\frac{16+4+6+17}{4} = 10.75 \text{ ms}$

5. Preemptive Priority (SOLVED)

Problem: Compute Avg W_t and Avg T_{at} using Preemptive Priority for the above table.

Solution:

- **Timeline Trace:**
  - $t = 0$: P1, P2 arrive. P2 (Pri 7) runs.
  - $t = 1$: P4 arrives (Pri 1). P2 continues.

- $t = 3$: P3 arrives (Pri 9). P3 preempts P2. P2 has 1ms remaining. P3 runs.
- $t = 8$: P3 finishes. P2 resumes.
- $t = 9$: P2 finishes. P1 (Pri 4) runs.
- $t = 16$: P1 finishes. P4 (Pri 1) runs.
- $t = 18$: P4 finishes.

- **Wait Time** = (Start - Arrival) + $\sum(\text{Resume} - \text{Preempt})$
  - $W_{t1} = 9 - 0 = 9$
  - $W_{t2} = (0 - 0) + (8 - 3) = 5$
  - $W_{t3} = 3 - 3 = 0$
  - $W_{t4} = 16 - 1 = 15$
  - **Average** W_t = $\frac{9+5+0+15}{4} = 7.25 \text{ ms}$
- **Turnaround Time** = $W_t + \text{Burst}$
  - $T_{at1} = 9 + 7 = 16$
  - $T_{at2} = 5 + 4 = 9$
  - $T_{at3} = 0 + 5 = 5$
  - $T_{at4} = 15 + 2 = 17$
  - **Average** T_{at} = $\frac{16+9+5+17}{4} = 11.75 \text{ ms}$

6. Round Robin (RR) (SOLVED)

Problem: Use RR (Time Quantum = 2ms) to compute Avg T_{at} and W_t.

| **Process** | **Arrival Time** | **CPU-burst-time** |

| P1 | 0 | 4 |

| P2 | 1 | 5 |

| P3 | 2 | 3 |

Solution:

- **Timeline Trace:**

- $t = 0 \to 2$: P1 runs. P1 rem = 2. (Queue: P2, P3, P1)
- $t = 2 \to 4$: P2 runs. P2 rem = 3. (Queue: P3, P1, P2)
- $t = 4 \to 6$: P3 runs. P3 rem = 1. (Queue: P1, P2, P3)
- $t = 6 \to 8$: P1 runs. P1 finishes. (Queue: P2, P3)
- $t = 8 \to 10$: P2 runs. P2 rem = 1. (Queue: P3, P2)
- $t = 10 \to 11$: P3 runs. P3 finishes. (Queue: P2)
- $t = 11 \to 12$: P2 runs. P2 finishes.

- **Turnaround Time** (T_{at} = Finish Time - Arrival Time)
  - $T_{at1} = 8 - 0 = 8$
  - $T_{at2} = 12 - 1 = 11$
  - $T_{at3} = 11 - 2 = 9$
  - **Average** T_{at} = $\frac{8+11+9}{3} = 9.33 \text{ ms}$
- **Wait Time** ($W_t = T_{at} - \text{Burst}$)
  - $W_{t1} = 8 - 4 = 4$
  - $W_{t2} = 11 - 5 = 6$
  - $W_{t3} = 9 - 3 = 6$
  - **Average** W_t = $\frac{4+6+6}{3} = 5.33 \text{ ms}$

Day 4: Process Commands, Vi Editor & Compilation

Ex 1 & 2: Finding Processes

- Type tty to find out your terminal number.
- Type ps to observe processes in the current terminal.
- Type ps -e to observe processes in the entire system.

Ex 3 & 4: Vi Editor Basics

- Open editor: vi hello.txt
- Press i (insert mode), type, press Esc (command mode).
- Save and quit: :wq
- View file: cat hello.txt or tac hello.txt
- Remove file: rm hello.txt

Ex 5 & 6: Recursive Listing and Folder Deletion

- See contents of subfolders: ls -R
- Remove an empty folder: rmdir notes
- Remove a non-empty folder recursively: rm -r practicals

Ex 7: C Compilation

- Write hello.c.
- Compile: gcc -o ./cprog/h ./cprog/hello.c
- Run: ./cprog/h

Ex 8, 9, 10 & 11: System Calls & PIDs

- View manual: man ls, man getpid (press :q to quit).
- Copy file: cp ./cprog/hello.c ./cprog/seepid.c
- Get process ID programmatically: getpid()
- Get parent process ID programmatically: getppid()

Day 5: Basic Shell Scripting & Process Forking

Ex 12: Simple Script

```
# File: ex1.sh
echo "hello"
# Run with: bash ./shells/ex1.sh
```

Ex 13: Taking Input

```
# File: ex2.sh
echo "enter your name "
read name
echo "hello $name"
```

Ex 14: Difference between echo and echo -n (SOLVED)

Problem: What does echo -n do differently?

Solution: The standard echo command automatically appends a newline character at the end of the output, pushing the next output to a new line. The -n flag suppresses this trailing newline, causing the subsequent text to be printed on the same continuous line.

Ex 15: Create Directory from Input

```
echo "enter a name of folder"
```

```
read foldername
mkdir ~/$foldername
echo "CHECK the contents of ~ "
ls ~
```

Ex 16: Check if File Exists

```
echo "enter the file path "
read filepath
if [ -f $filepath ]; then
  cat $filepath
else
  echo "$filepath does not exist"
fi
```

Optional Fork Assignment

```
#include<stdio.h>
#include<unistd.h>
#include<sys/types.h>
#include<wait.h>
void main() {
  int child_pid;
  child_pid=fork();
  printf("pid=%d, ppid=%d\n", getpid(), getppid());
  waitpid(child_pid, NULL, 0); // parent waits for child
}
```

Day 6: Shell Scripting Arithmetic & Text Handling

Ex 1: Integer Addition (expr)

```
echo "enter 2 numbers"
read num1 num2
expr $num1 + $num2
```

Ex 2: Backquotes & Variables

```
echo "enter 3 numbers"
read n1 n2 n3
sum=`expr $n1 + $n2 + $n3`
echo "sum=$sum"
average=`expr $sum / 3`
echo "average=$average"
```

Ex 3 & Floating Point Calculation

```
echo "enter 2 numbers"
read n1 n2
echo "scale=2; $n1 + $n2" | bc
```

Ex: String Length (SOLVED)

Problem: The provided note code calculates string length but subtracts 1 because wc -c counts the invisible newline character.

Alternative Clean Solution:

```
echo "enter a string"
read str
len=${#str} # Native bash string length measurement
echo "length of $str is $len"
```

Ex: Redirection & Sorting (SOLVED)

Problem: Write a shell script to accept many numbers from a user, save it in a file, and show the sorted numbers in ascending order.

Solution:

```
echo "Enter numbers (Press Ctrl+D when finished):"
cat > numbers.txt
echo "--- Sorted Numbers ---"
sort -n numbers.txt
```

Day 7: Regex, Permissions & Command Line Arguments

Regex Examples (grep -E)

- Match exactly 10 digits: [0-9]{10}
- Match month (Title Case, 3 letters): [JFMASOND][a-z]{2}
- Match Email: ^[A-Za-z][A-Za-z0-9]+@[a-z]+\.[a-z]{2,3}$

Permissions Practice (chmod)

- Remove read from user: chmod u-r hi.txt
- Add read/write to all: chmod ugo=rw hello.sh
- Octal absolute permissions: chmod 764 hello.sh (User=rwx, Group=rw, Other=r)

Command Line Arguments Example (Ref: cmdlineEx.png)

The provided screenshot cmdlineEx.png demonstrates how positional parameters ($1, $2, etc.) and the shift command work inside a bash script to parse arguments iteratively:

```
sum=0
while [ $# -gt 0 ]
do
 echo "\$1 = $1"
 sum=`echo "$sum+$1" | bc`
 shift
done
echo "sum= $sum"
```

Practice Problems (SOLVED)

Problem 1: Password Validation Regex

Task: Validate a password that must begin with an alphabet, have a max of 8 characters, and contain only alphabets, numbers, or _, -. Print "valid" or "invalid".

Solution:

```
echo "Enter password:"
read pass

# Max 8 chars total means 1 starting char + max 7 following chars
if echo "$pass" | grep -E -q "^[a-zA-Z][a-zA-Z0-9_-]{0,7}$"; then
```

```
    echo "Valid password"
else
    echo "Invalid password"
fi
```

Problem 2: Menu Driven Script

Task: Create a menu to manage directories based on user choice.

Solution:

```
while true; do
    echo -e "\na. Create directory"
    echo "b. Check if directory is present"
    echo "c. Remove directory"
    echo "d. Show content of directory"
    echo "e. Quit"
    read -p "Enter your choice: " choice

    if [ "$choice" == "e" ]; then
        echo "Exiting..."
        break
    fi

    read -p "Enter the absolute path of the directory: " dirpath

    case $choice in
        a)
            mkdir -p "$dirpath"
            echo "Directory created."
            ;;
        b)
            if [ -d "$dirpath" ]; then
                echo "Directory is present."
            else
                echo "Directory not found."
            fi
            ;;
        c)
            rm -r "$dirpath"
            echo "Directory removed."
            ;;
```

```
      d)
        ls -R "$dirpath"
        ;;
      *)
        echo "Invalid choice. Please try again."
        ;;
  esac
done
```

Day 8: Page Replacement Algorithms

Context: Page Access String = 1, 2, 3, 1, 4, 2, 5, 4. Number of available frames = 3.

1. FIFO (First In First Out)

Solution Summary:

- Total Page Faults = 5
- Total Replacements = 2

2. LRU (Least Recently Used)

Solution Summary:

- Total Page Faults = 6
- Total Replacements = 3

3. MRU (Most Recently Used)

Solution Summary:

- Total Page Faults = 5
- Total Replacements = 2

4. Second Chance Algorithm

Context: Page Access String = 0, 4, 1, 4, 2, 4, 3, 0. Number of frames = 3.

Format: PageNumber(ReferenceBit)

- F1: 0(1) -> 0(1) -> 0(1) -> 0(0) -> 2(1) -> 2(1) -> 2(1) -> 0(1)
- F2: E -> 4(1) -> 4(1) -> 4(0) -> 4(0) -> 4(1) -> 4(0) -> 4(0)
- F3: E -> E -> 1(1) -> 1(0) -> 1(0) -> 1(0) -> 3(1) -> 3(1)
- *Observation: If a page is selected for replacement but its reference bit is 1, it gets a "second chance" (bit reset to 0) and the pointer moves to the next oldest page.*